UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Estudios Superiores

ACTIVIDAD 1

Sistema generador de números pseudo aleatorios usando microcontrolador ATMEGA 328p

Autores
Luna Leyva Azul Desiree
Ramos Ramírez Alberto Carlos
Santana Martínez Josué Eleazar

10 de marzo de 2026

Resumen

Los algoritmos de ordenamiento son procedimientos lógicos para organizar un conjunto de datos en un orden específico (ascendente o descendente). En ciencias de la computación, su eficiencia se mide con la notación Big-O, que describe cómo crece el tiempo de ejecución a medida que aumenta el número de datos.

Introducción

La programación en lenguaje ensamblador permite una interacción directa con la arquitectura del CPU, facilitando el control preciso sobre módulos como la ALU, los registros de propósito general y la memoria SRAM.

En el contexto de los sistemas embebidos, la eficiencia en el manejo de datos es crítica debido a las limitaciones de recursos de hardware.

El objetivo principal de esta práctica es desarrollar capacidades para la escritura de código de bajo nivel que manipule estructuras de datos en memoria.

Para ello se utilizan punteros de memoria y ciclos de control que permiten transformar una tabla de números desordenados en listas organizadas bajo criterios específicos.

Teoría e importancia de los algoritmos de ordenamiento

El ordenamiento de datos consiste en organizar un conjunto de elementos siguiendo una relación de orden (numérica o alfabética).

Su importancia radica en la optimización de otros procesos: un conjunto de datos ordenado reduce drásticamente el tiempo necesario para realizar búsquedas, permitiendo el uso de búsqueda binaria.

En sistemas con microcontroladores elegir el algoritmo correcto es vital, ya que algunos priorizan el ahorro de memoria RAM mientras que otros reducen los ciclos de instrucción.

Notación Big-O

Es una herramienta fundamental en la programación para medir la eficiencia de un algoritmo.

Evalúa el tiempo y el uso de memoria en función del tamaño de entrada, permitiendo comparar algoritmos y optimizar el rendimiento.

No mide el tiempo exacto en segundos, sino cómo crece el número de operaciones al aumentar el tamaño de los datos.

Algoritmo	Complejidad	Comparaciones	Ciclos de reloj	Tiempo (μs)
Bubble Sort	$O(n^2)$	4950	112540	112538
Selection Sort	$O(n^2)$	4950	44383	43418

Cuadro 1: Comparación entre algoritmos

Pruebas realizadas

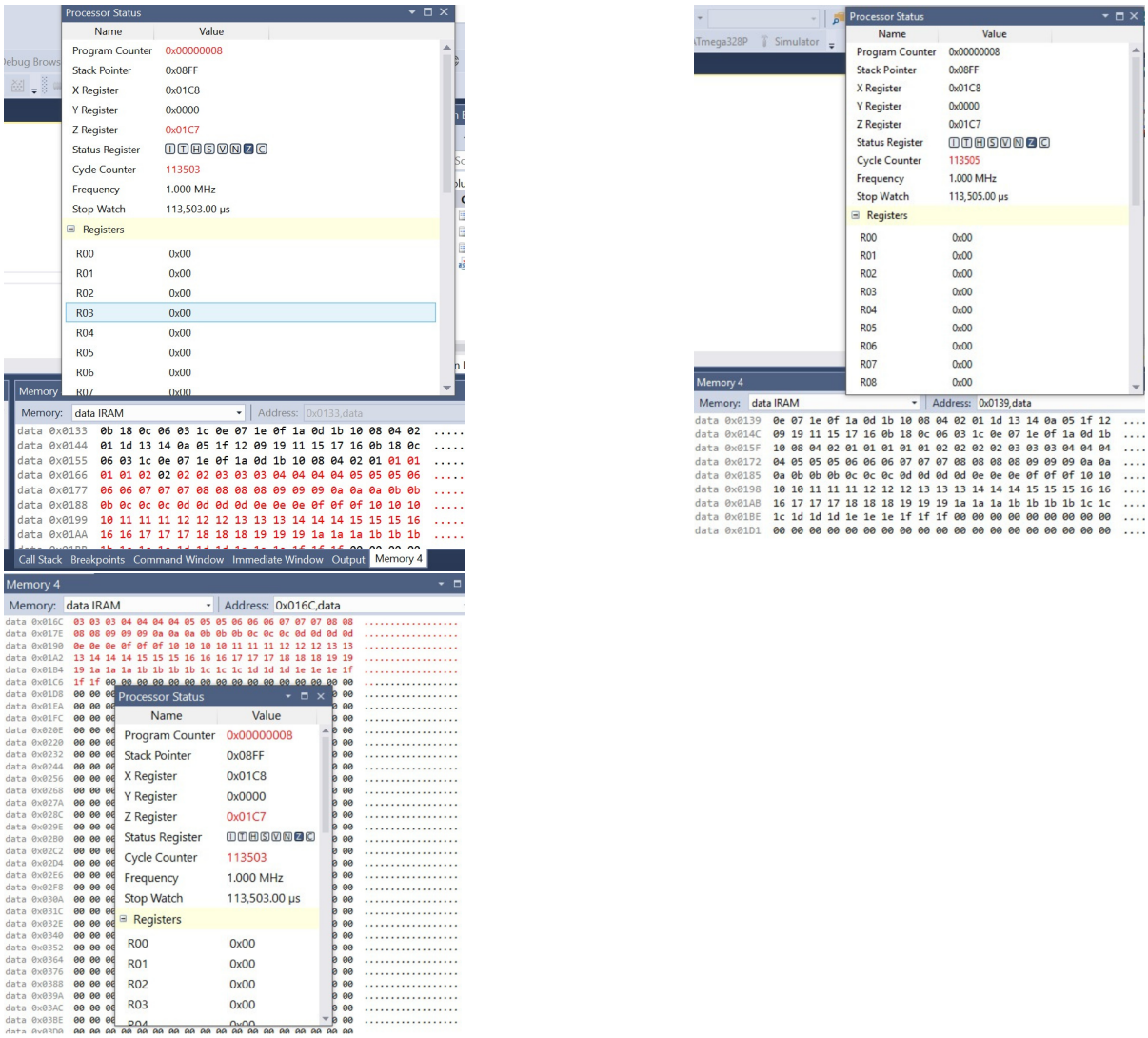

Figura 1: Prueba 01, 02 y 03. Código 1

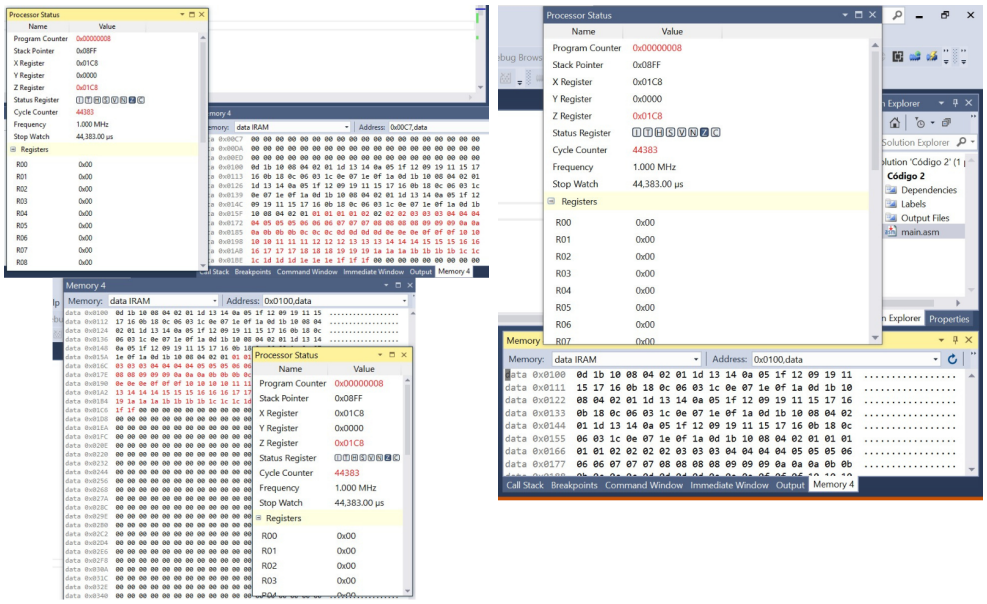

Figura 2: Prueba 01, 02 y 03. Código 2

Tabla de resultados

Parámetro	Bubble Sort	Selection Sort	Observaciones
Complejidad	$O(n^2)$	$O(n^2)$	Crecimiento cuadrático
Comparaciones	4950	4950	$2n(n-1)$
Ciclos de reloj	113505	44383	Selection es más rápido
Tiempo (μs)	113505	44383	Frecuencia 1 MHz

Análisis de resultados

Aunque la notación Big-O es la misma para ambos algoritmos, el Selection Sort terminó la tarea en menos de la mitad del tiempo.

Esto se debe a que Bubble Sort realiza intercambios constantes en memoria, lo que consume más ciclos de instrucción en lenguaje ensamblador.

Pseudocódigo

Código 01: Bubble Sort

```
ALGORITMO: Ordenamiento_Burbuja

ENTRADA: tabla_de_numeros_desordenados (100 elementos)
SALIDA: tabla_ordenada

INICIO

Generar números aleatorios
```

```
Copiar datos a tabla

Para i = 0 hasta N-2
    Para j = 0 hasta N-i-2

        Si memoria[j+1] < memoria[j]
            intercambiar valores
        Fin si

    Fin para
Fin para

FIN
```

Código 02: Selection Sort

```
ALGORITMO: Ordenamiento_por_Seleccion

ENTRADA: tabla_de_numeros_desordenados
SALIDA: tabla_ordenada

INICIO

Generar números aleatorios
Copiar datos a tabla

Para posicion_actual desde inicio hasta penúltimo

    valor_minimo = dato[posicion_actual]

    Para j desde posicion_actual+1 hasta final

        Si dato[j] < valor_minimo
            actualizar minimo
        Fin si

    Fin para

    intercambiar valores

Fin para

FIN
```

Conclusiones

Luna Leyva Azul Desireé

En esta práctica fuimos capaces de entender cómo es que funcionan los módulos de la EEPROM y RAM, y de esta forma comprender de manera práctica como es que se gestionan los datos a nivel de hardware haciendo uso de la ATMEGA 328p en lenguaje ensamblador

Ramos Ramírez Alberto Carlos

Pra el diseño de sistemas embebidos que se puede pensar que son limitados por los resultados obtenidos, el Selection Sort es preferible sobre el Bubble Sort. A pesar de compartir la misma complejidad teórica, la reducción en las operaciones de escritura en memoria lo convierte en una solución más robusta y rápida para el procesamiento de datos internos.

Santana Martínez Josué Eleazar

Aunque ambos algoritmos tienen la misma clasificación teórica, las pruebas en el depurador demostraron que el Selection Sort es significativamente más rápido, completando la tarea en 44,383 ciclos frente a los 113,505 ciclos del Bubble Sort.

Referencias

- `https://www.freecodecamp.org/espanol/news/explicacion-de-la-notacion-big-o-con-e`
- `https://developero.io/blog/big-o`
- `https://elblogdelprogramador.com/posts/big-o-notation-que-es-y-por-que-es-import`